

Unit 2 – RPC, Distributed Objects & Communication (Simplified Detailed Notes)

These notes explain every topic and subtopic from Unit 2 in a detailed yet easy-to-understand way. Difficult concepts are simplified using step-by-step explanations, real-life examples, and comparisons so they are easier to learn for exams.

Introduction to Remote Procedure Call (RPC)

Remote Procedure Call (RPC) allows a program on one computer to execute a function on another computer as if the function were local.

Main Idea:

The programmer writes code normally, while RPC hides networking complexity.

Example:

Suppose a food delivery app running on your phone needs restaurant data from another server.

Instead of manually handling:

- sockets
- packets
- network protocols

The app simply calls:

`getRestaurantDetails()`

The actual function runs on another machine, but it feels like a normal function call.

RPC is important because:

- It simplifies distributed programming
- Hides communication details
- Makes remote communication easier

Problems in RPC

Although RPC looks like a local function call, many challenges exist internally.

1. Different Memory Spaces

Client and server run on different machines.

Pointers cannot be shared directly.

2. Different Data Formats

Different computers may store data differently.

Example:

- Big-endian
- Little-endian

3. Network Delays

Remote calls are much slower than local calls.

4. Failures

Problems may occur:

- Network failure
- Server crash
- Client crash

5. Security Issues

Messages may be intercepted or modified.

6. Naming Problem

Client must locate:

- Which server?
- Which procedure?

Therefore:

RPC systems need directory and naming services.

Conventional Procedure Call

A normal procedure call happens inside the same machine.

How it works:

1. Caller places parameters in memory stack.
2. Control transferred to called function.
3. Function executes.
4. Result returned.

Because both functions share memory:

Procedure calls are extremely fast.

Example:

sum(5, 10)

The CPU directly jumps to the function.

Parameter Passing Methods:

1. Call-by-Value

A copy of variable passed.

Changes inside function do not affect original variable.

Example:

```
x = 5
```

```
modify(x)
```

Even if modify changes x to 10:

Original x remains 5.

2. Call-by-Reference

Address of variable passed.

Changes affect original variable.

3. Call-by-Copy/Restore

Copy passed initially.

Modified copy returned later.

Client and Server Stubs

RPC uses special helper programs called stubs.

Purpose:

Hide networking details.

1. Client Stub

Acts like local function.

Responsibilities:

- Accept function call
- Pack parameters (marshalling)
- Send request to server
- Wait for reply

2. Server Stub

Acts as intermediary on server.

Responsibilities:

- Receive request
- Unpack parameters
- Call actual server procedure
- Pack result
- Send reply

Example:

Client thinks:

calculateBill()

Actually:

Client stub sends network message to remote machine.

Working of RPC Using Stubs

Step-by-step flow:

1. Client calls local stub.
2. Client stub marshals parameters.
3. Message sent across network.
4. Server stub receives message.
5. Server stub unmarshals parameters.
6. Actual server function executes.
7. Result returned to server stub.
8. Server stub sends reply.
9. Client stub unmarshals result.
10. Result returned to client program.

To the programmer:

Everything appears like a normal function call.

RPC Runtime

RPC runtime is the communication layer below stubs.

Responsibilities:

1. Network Transmission

Transfers request/reply messages.

2. Retransmission

Resends lost messages.

3. Acknowledgments

Confirms message delivery.

4. Packet Routing

Ensures packets reach destination.

5. Encryption

Protects messages.

Example:

If network packet lost:

RPC runtime retransmits automatically.

Marshalling and Unmarshalling

Marshalling:

Converting data into network message format.

Unmarshalling:

Converting received message back into usable data.

Why needed?

Different machines represent data differently.

Example:

Machine A:

Little-endian

Machine B:

Big-endian

Without conversion:

Values may become incorrect.

Real-Life Example:

Like packing goods into standardized shipping boxes before transport.

Parameter Passing in RPC

Suppose remote procedure:

`k = add(i, j)`

Working:

1. Client calls `add(i,j)`.
2. Stub packs:
 - Procedure name
 - Parameter values
3. Message sent.
4. Server unpacks values.
5. `add()` executes remotely.
6. Result returned.

Issues:

Machines may differ in:

- Integer size
- Floating-point format
- Byte order
- Structure alignment

Therefore:

RPC systems define standard data representation.

Passing Reference Parameters

Passing pointers in RPC is difficult.

Why?

Pointers only make sense inside original process memory.

Example:

Client pointer:

`0xABC123`

That memory address may not exist on server.

Solution:

Copy/Restore Technique

Steps:

1. Client copies data into message.
2. Server creates local copy.
3. Procedure works on local copy.
4. Modified data copied back.

Advantages:

Works well for:

- Arrays
- Simple structures

Limitations:

Poor for:

- Graphs
- Linked lists
- Complex objects

Parameter Specification and Stub Generation

Client and server must agree on:

- Message format
- Parameter representation
- Communication protocol

This set of rules forms RPC protocol.

Example:

Both sides must agree:

- Integer size
- Character encoding
- Float representation

Interface Definition Language (IDL)

IDL defines remote procedures formally.

IDL compiler automatically generates:

- Client stubs
- Server stubs
- Support code

Benefits:

- Reduces manual coding
- Prevents communication mismatch

Distributed Objects

Distributed objects are objects located on remote machines but accessible like local objects.

An object contains:

- State (data)
- Methods (functions)

Example:

Online banking system:

Account object stored on server.

Client invokes:

deposit()

withdraw()

As if object were local.

Proxy and Skeleton

Distributed object systems use:

1. Proxy (Client Side)

Acts like remote object locally.

Responsibilities:

- Marshal method calls
- Send requests

2. Skeleton (Server Side)

Receives requests.

Responsibilities:

- Unmarshal data
- Invoke actual method
- Return result

Only method invocations cross network.

Actual object state stays on server.

Compile-Time vs Runtime Objects

Compile-Time Objects:

- Stubs generated before execution
- Language dependent

- Less flexible

Example:

Java RMI

Runtime Objects:

- Binding happens dynamically
- More flexible
- Language independent

Example:

Middleware systems

Comparison:

Compile-time → faster but rigid

Runtime → flexible but complex

Persistent and Transient Objects

Persistent Objects:

Continue existing after server stops.

State stored on:

- Disk
- Database

Example:

Database records

Advantages:

- Survive crashes
- Higher reliability

Transient Objects:

Exist only while server running.

Stored only in memory.

Example:

Temporary chat session

If server crashes:

Object lost.

Client–Object Binding

Before client invokes remote object:
Binding must occur.

Binding:
Connecting client to distributed object.

Object References:
Instead of memory addresses:
Clients use object references.

Benefits:
- Location transparency
- Easier distributed communication

Types:

1. Implicit Binding
Occurs automatically.
2. Explicit Binding
Client manually binds before use.

Remote Method Invocation (RMI)

RMI allows clients to invoke methods on remote objects.

Working:

1. Client calls method on proxy.
2. Proxy marshals request.
3. Server activates object.
4. Skeleton unmarshals request.
5. Method executes.
6. Result returned.

RMI extends RPC by supporting objects.

RMI Parameter Passing

1. Passing Object References
Easy because server can directly invoke methods on referenced object.

2. Passing Objects by Value

Entire object copied and transmitted.

Includes:

- Data
- Method references

Server creates copy after unmarshalling.

Message-Oriented Communication

Instead of invoking procedures:

Processes exchange messages.

Main Feature:

Communication may be asynchronous.

Advantages:

- Receiver need not be active
- Loose coupling
- Better scalability

Message-Oriented Transient Communication

Messages delivered only if receiver active.

No persistent storage.

If receiver unavailable:

Message lost.

Example:

Live chat without offline message storage.

Berkeley Sockets

Sockets provide standard API for network communication.

Server Side Steps:

1. Create socket
2. Bind IP and port
3. Listen
4. Accept connection

5. Send/receive data
6. Close connection

Client Side:

1. Create socket
2. Connect to server
3. Send/receive
4. Close

Example:

Web browser communicating with website server.

Message Passing Interface (MPI)

MPI is used in parallel computing.

Designed for:

- Supercomputers
- Clusters

Advantages:

- High performance
- Efficient communication
- Portable

Features:

- Blocking communication
- Non-blocking communication

Example:

Weather simulation systems.

Persistent Communication

Messages stored until delivered.

Sender and receiver need not run simultaneously.

Implemented using:

- Message queues
- Message-oriented middleware (MOM)

Advantages:

- Reliable

- Asynchronous
- Decoupled communication

Message Queueing Model

Applications communicate through queues.

Example:

Email system.

Sender sends mail even if receiver offline.

Message stays stored until receiver accesses it.

Execution Cases:

1. Both active
2. Sender active only
3. Receiver active only
4. Both inactive

Queues support all cases.

Asynchronous Communication

Sender does not wait after sending.

Advantages:

- Faster
- Better parallelism

Example:

WhatsApp sending message while user continues typing.

Synchronous Communication

Sender waits after sending.

Waiting may continue until:

- Middleware accepts message
- Receiver receives message
- Receiver processes message

Example:

Phone call conversation.

Transient vs Persistent Communication

Transient Communication:

- Temporary
- Receiver must be active
- Less reliable

Examples:

Sockets, MPI

Persistent Communication:

- Messages stored
- Receiver may be offline
- Higher reliability

Examples:

Message queues, email systems

Stream-Oriented Communication

Deals with continuous flow of data.

Examples:

- Audio streaming
- Video streaming
- VoIP calls

Difference from message-oriented systems:

Timing is critical.

Late packets may:

- Distort audio
- Freeze video

Data Streams

A data stream is sequence of data units.

Types:

1. Discrete Media

- Files

- TCP data

2. Continuous Media

- Audio
- Video

Continuous streams require strict timing.

Transmission Modes

1. Asynchronous Transmission

No strict timing.

Example:

File transfer

2. Synchronous Transmission

Maximum delay allowed.

Example:

Sensor systems

3. Isochronous Transmission

Precise timing required.

Example:

Video conferencing

Simple and Complex Streams

Simple Stream:

Single data sequence.

Example:

Audio file

Complex Stream:

Multiple synchronized streams.

Example:

Movie:

- Video
- Audio
- Subtitles

All streams must remain synchronized.

Streaming Stored Media

Multimedia server stores compressed media.

Server:

- Controls QoS
- Sends streams

Client:

- Decodes streams
- Synchronizes playback

Goal:

Maintain smooth multimedia playback.

Quality of Service (QoS)

QoS defines performance guarantees for streams.

Main QoS Goals:

1. Timeliness
2. Bandwidth
3. Reliability

Important Parameters:

- Bit rate
- Delay
- Jitter
- Round-trip delay

Example:

Netflix video streaming requires stable QoS.

Token Bucket Algorithm

Used for traffic control and QoS.

Working:

- Tokens added at constant rate
- Sending packet requires token

If no token:

Packet waits or discarded.

Advantages:

Allows bursts while controlling average rate.

Example:

Internet bandwidth control.

Streaming and Synchronization

Synchronization keeps related streams aligned.

Examples:

- Audio with video
- Slides with narration

Types:

1. Discrete-Continuous

Slides + audio

2. Continuous-Continuous

Audio + video

Lip synchronization is important example.

Synchronization Mechanisms

Synchronization may happen at low level.

Example:

For every video frame:

Play two audio chunks.

Purpose:

Keep streams aligned.

This is explicit synchronization.

High-Level Stream Synchronization

Handled by multimedia middleware.

Application only specifies:
What should happen.

Middleware handles:

- Audio/video timing
- Device control
- Synchronization

Advantages:

- Easier programming
- Better abstraction
- Automatic synchronization

Exam Tip: Focus on understanding the flow of communication step-by-step. Topics like RPC, stubs, message queues, MPI, QoS, and synchronization become much easier when you imagine real-life systems such as WhatsApp, Netflix, banking servers, and cloud applications.